Detection and Analysis of Dysfluency in Speech

Ajay Shastry C G, K. Shreekrishna Upadhyaya, M. Chethan Keshav Bhat, Skanda Prasad K, and Srinivas Hegde M
Department of Artificial Intelligence & Machine Learning,
Vivekananda College of Engineering & Technology,
Puttur, Karnataka, India
{ajayshastrycg.ai, 4vp23ai020, 4vp23ai023, 4vp23ai051, 4vp23ai054}@vcetputtur.ac.in

***Abstract*—** Stuttering-related dysfluencies vary substantially in acoustic form and are strongly class-imbalanced, making reliable auto matic detection difficult. This paper presents a lightweight end-to-end system that detects five dysfluency types using independently trained binary convolutional neural networks over 3 s, 16 kHz log-mel spectro grams. A shared inference module aggregates per-class chunk scores and drives a desktop interface. On a held-out set of 4,193 clips, a fixed 0.5 threshold yields a macro positive-class F1 of 0.382, rising to 0.467 after per-class calibration, with word repetition collapsing entirely onto the majority class. Reporting prevalence-matched trivial baselines alongside these results, the paper positions the system as a compact, transparent baseline and identifies annotation conversion, class imbalance, and tem poral context as the most promising directions for improvement.

Keywords—Speech dysfluency detection, stuttering classification, speech processing, log-mel spectrograms, convolutional neural networks, class imbalance, model evaluation, reproducibility

I. INTRODUCTION

Stuttering affects approximately 1% of the global adult population and up to 5% of children [1]. It is characterized by involuntary repetitions of sounds, syllables, or words (sound repetitions and word repetitions), abnormal prolongations of speech sounds (prolongations), and involuntary pauses or stops in speech flow (blocks), and it degrades communication and the psychosocial well-being of affected individuals [1]. Clinical assessment involves expert perceptual judgment and can require substantial time and effort, motivating computational tools that may assist with consistent analysis outside controlled clinical workflows.

Two obstacles stand between that clinical practice and an automated system. The first is representational: the five dysfluency types have different and individually subtle acoustic signatures, and they are rare in ordinary speech, so most segments of a recording carry no event at all. The second is operational: a detector is only useful if a non-specialist can run it on an arbitrary recording, on ordinary hardware, and read a result they can interpret. Published models are usually reported as tables on a held-out split; the path from such a table to a running tool is left unspecified, and the trivial predictors that a heavily imbalanced task invites are rarely reported alongside.

This paper attacks both obstacles with one compact mechanism and reports it against those trivial baselines, including on the arms where it loses to them. We build five independent binary convolutional classifiers over log-mel spectrograms, one per dysfluency type, and specify the whole inference path (chunking, per-model feature extraction, scoring, cross-chunk aggregation, thresholding, and presentation) as a shared module that the PyQt5 desktop application uses unchanged.

The paper makes the following contributions:

1. We present a five-model binary CNN detection system for speech dysfluency: one independently trained classifier per dysfluency type, each with 101,441 parameters, built from a three-block convolutional encoder over a 3 s, 16 kHz mono log-mel spectrogram. The dataset tooling that slices annotated corpora into fixed windows ships with it.

2. We specify the deployed inference path in full: non-overlapping 3s chunking, filename-encoded per-model mel configuration, sigmoid scoring, maximum-probability aggregation across chunks, and a two-valued decision at a fixed threshold of 0.45, driven by a PyQt5 desktop client over one shared inference module.

3. We report a transparent held-out evaluation of 4,193 clips using positive-class precision, recall, F1, accuracy, and full confusion counts, together with prevalence-matched always-positive and always-negative baselines. This exposes both useful signal and degenerate majority-class behaviour.

4. We analyze three experimentally testable contributors to the observed failure modes: OR-based conversion of annotator votes, unweighted binary cross-entropy under class imbalance, and limited temporal context from fixed non-overlapping 3 s windows.

5. We characterize the computational profile of the complete five-model pipeline, showing that the compact architecture can process 3 s windows in real time on CPU-only hardware.The remainder of this paper is organized as follows: Section II reviews related work and states the research gap, Section III describes the proposed system, Section IV details the implementation, Section V presents results and discussion, and Section VI concludes.

II. RELATED WORK

Traditional Approaches

Early stutter detection systems relied on handcrafted acoustic features, including jitter, shimmer, spectral flux, and formant trajectories, combined with classical classifiers such as Support Vector Machines (SVMs), Hidden Gaussian Models, and Gaussian Mixture Models. These methods achieved reasonable accuracy on controlled datasets but generalized poorly across speakers, accents, and recording conditions, because handcrafted features have limited representational

capacity and because the acoustic cues of a block or a prolongation overlap with those of ordinary pausing and slow articulation.

Deep Learning Methods

The introduction of end-to-end deep learning removed the need for manual feature engineering. Convolutional Neural Networks (CNNs) operating on mel-spectrograms showed early promise for dysfluency detection, and remain the dominant choice when the deployment target is CPU-only inference. Recurrent networks and Long Short-Term Memory (LSTM) layers were applied to capture temporal dependencies in speech signals. The SEP-28K dataset and its extension SEP-28K-E provide tens of thousands of podcast clips annotated for five dysfluency types and an event-localization benchmark [2] FluencyNet applied deep residual networks to stutter disfluency detection and introduced the LibriStuttercorpus [3], and StutterNet established the time-delay neural network formulation that remains the common CPU-side baseline [4]. Work on class imbalance is directly relevant to our two failing arms: positive-weighted and class-balanced objectives have been reported to help precisely on the minority stuttering types [5]. Context-aware and transcript-free variants of the same detection task have also been explored [6], [7], and multimodal systems that fuse acoustic with linguistic or visual features report further ga ins [8]. A large body of subsequent work treats clip-level presence of a dysfluency type as a multi-label binary problem, one decision per type per clip, which is the formulation adopted here [9].

Most such studies train a single multi-label network over all types jointly.We train one small binary network per type, which costs a factor of five in parameters and keeps each decision independent and separately replaceable. The multimodal systems in particular are an existence proof that the remaining headroom on this task is not exhausted by audio-only modelling.

Self-Supervised and Transformer-Based Detection

Transformer-based architectures, particularly Wav2Vec 2.0 [10], learn contextualized representations from raw audio through self-supervised pre-training and have achieved strong results on downstream speech tasks. Fine-tuning Wav2Vec2 for dysfluency detection was established on therapy and read speech [11]. Bayerl et al. applied Wav2Vec2 to stuttering detection as an explicitly multi-label problem across three corpora, including FluencyBank, and reported a macro F1 of 0.56 over five dysfluency types on FluencyBank [9], [12] the large-language-model extension of the same line reaches 0.61 [13]. Their per-class profile is close to the inverse of ours: word repetition is their strongest class at 0.64 and block their weakest at 0.32, whereas word repetition is the class we fail on entirely and block the one we fail by the widest margin. The two systems therefore exhibit different class-wise failure profiles. This suggests that the weak classes in our system are not necessarily intrinsically undetectable, while not establishing whether representation, temporal context, or label processing is responsible. Alternative self-supervised encoders such as HuBERT [14], WavLM [15], and XLS-R [16] capture acoustic and cross-lingual representations from two to three orders of magnitude more parameters than the encoder used here. We do not evaluate that family. A 94M-parameter encoder is substantially larger than the compact CPU-oriented design targeted here, and the purpose of this paper is to specify and measure a small end-to-end mechanism rather than to re-run that comparison. Section V situates our numbers against the published figures without claiming a matched comparison, and Section V lists the encoder study as required future work.

Evaluation Practice and Reporting

A recurring difficulty in this literature is that dysfluency detection is evaluated on splits whose class prevalences are high enough that trivial predictors score well: a rule that answers "positive" for every clip attainsan F1 of 2 p at prevalence p, and a rule that answers "negative" attains $1+p$ an accuracy of $1 - p$. Reported accuracy on such a split is therefore uninformative on its own, and a per-class F1 that is not accompanied by the prevalence-matched baseline cannot be interpreted. Degenerate solutions are the extreme case: a classifier that never predicts the positive class reports a plausible accuracy equal to the majority rate and an F1 of zero, and nothing in a bare accuracy column distinguishes it from a working detector. We attach both trivial predictors to every result table in Section V, and we treat a model that does not exceed them as a negative result rather than as a ranking.

Research Gap

Two gaps follow from the reviewed work. The first is a reporting gap: published dysfluency-detection results are stated as headline accuracy or macro F1 on splits of undisclosed class prevalence, and are rarely accompanied by the prevalence-matched and degenerate predictors that a per-class figure needs in order to be interpretable at all. The second is an attribution gap. The one published system that reports per-class scores for these same five types exhibits a failure profile close to the inverse of ours, which establishes that the classes we fail on are not intrinsically undetectable but leaves unresolved whether the failure originates in the acoustic representation, in the temporal context given to the decision, or in the production of per-class scores and thresholds. No reviewed study separates these three causes, because doing so requires independent per-type decisions rather than one joint multi-label model. This paper addresses both gaps by training five independent per-type classifiers, reporting both trivial baselines beside every result table, and designing the arms so that chunk aggregation is isolated from per-window scoring.

III. PROPOSED SYSTEM

The system has two parts: a classification pipeline that identifies which dysfluency types are present in speech audio, and a desktop front end that consumes it. The design goal is a compact, CPU-oriented research prototype whose training/evaluation path and application inference path share the same model architecture and preprocessing logic. The pipeline builds on a common preprocessing module and loads models from configuration; the application imports the shared inference module directly, so the reported evaluation and the shipped application run bit-identical model code.

Data Pipeline

The mechanism is trained on SEP-28K [2], a corpus of roughly 28,000 three-second clips sliced from 385 publicly available podcast episodes featuring people who stutter, annotated with per-clip annotator vote counts over nine speech-disorder labels. Table I summarizes the corpora named by the project, distinguishing the one that the released training code actually consumes from the one that only the project documentation lists.

TABLE I. *Corpora named by the project and their actual role in the reported evaluation.The distinction matters for reproducibility: the released training notebook reads only the SEP-28K label file.*

Corpus	Clips	Setting	Annotation	Role in this study
SEP-28K [2]	28,177 label rows	Podcast episodes	Per-clip annotator votes (0–the released 3) over 9 labels	Consumed by training notebook
FluencyBank [3]	≈4,100 (documented)	TalkBank interviews	Same label scheme	Named in project documen-tation only; not read by the released code
Total used in the reported run	20,906	—	—	28,177 rows less 7,271 clips whose episode audio was unavailable

Corpus preparation is scripted. A shell entry point initializes the corpus submodule, verifies that ffmpeg is present, and calls a Python driver that downloads each episode, transcodes it to 16 kHz mono WAV with ffmpeg, and slices the annotated intervals into individual clip files; the download stage runs on a thread pool sized to the host (capped at eight workers), and the intermediate episode directory is deleted once slicing completes.The dataset is required only for retraining and is not needed to run the application.

Of the 28,177 label rows, 7,271 refer to episode audio that was not present in the working copy and were dropped, leaving 20,906 clips; 20,887 of these are exactly 3.0 s long and the remainder fall between 2.54 s and 2.90 s, so the analysis window truncates or zero-pads a small minority of clips. The split is a random 70/15/15 partition over clips, giving 4,193 held-out test clips under the clip-level split (or 4,008 under random split), the only evaluation partition for which per-class reports exist in the released notebook.

The label scheme needs a conversion step. SEP-28K records how many annotators (of about three) marked each clip with each label, and the released code reduces this to a binary target per type by taking the logical OR of the vote count: a clip is positive for a type if any annotator marked it. The sixth label in the notebook's label vector, no-speech, is computed but never modeled; the shipped system contains five classi fiers. Table II states the conversion explicitly, and the OR rule is one of the three candidate causes of the failure modes of Section V.

Features are extracted once and cached to disk as NumPy arrays, so that training epochs do not re-derive spectrograms. Each cached array is a dB-scaled mel spectrogram (power referenced to the clip maximum), z-score normalized with that clip's own mean and standard deviation. The normalization is per clip, so it removes absolute level information and makes the models insensitive to recording gain, which helps across devices and discards loudness as a cue. No augmentation of any kind is applied

Classification Pipeline

Five Independent Binary Classifiers:

Each dysfluency type is handled by a separate binary classifier over the log-mel spectrogram of a single 3 s window. The encoder is three convolutional blocks applied to a single-channel image: a 3×3 convolu tion from 1 to 32 channels, batch normalization, ReLU, and 2×2 max pooling; a 3×3 convolution from 32 to 64 channels with the same normal ization, activation, and pooling; and a 3×3 convolution from 64 to 128 channels followed by batch normalization, ReLU, and global average pooling to a 128-element vector. A fully connected head maps that vector through a 64-unit ReLU layer and a dropout of 0.3 to a single logit. The network holds 101,441 learned parameters including the batch-normal ization affine terms, and the count is independent of the mel resolution because global average pooling removes the spatial extent before the first dense layer.

Each classifier consumes its own mel configuration, which is encoded in the checkpoint filename under the convention {type}model{n_fft} {hop_length}{n_mels}_{epochs}.pth and parsed at load time. The released checkpoints use $n$fft = 1024, a hop length of 512, and either 64, 128, or 256 mel bands depending on the class, which yields spectrograms of shape (64, 94), (128, 94), or (256, 94) for a 48,000-sample window at 16 kHz. The filename also carries the epoch count of the run that produced the checkpoint. This convention removes a configuration file entirely: the artifact is self-describing, and a model cannot be loaded with a mel resolution that disagrees with the one it was trained on.

Cross-Chunk Scoring and Aggregation:

Inference operates on whole recordings. Audio is decoded to 16 kHz mono and cut into non-overlapping 3 s windows; the final window is zero-padded when the recording does not divide evenly. Each window is scored independently by all five

classifiers, which run concurrently on a five-thread pool so that the mel front end and the forward pass of one model overlap those of the others. Each model emits a sigmoid probability per window, and a class is flagged for that window when its probability exceeds the decision threshold.

A recording-level decision then takes the maximum probability per class over all windows, together with a count of flagged windows. We aggregate by maximum because one dysfluent window should be enough to raise the flag, and a mean would dilute a localized event across the windows that make up most of the recording. The reported detection score is that maximum expressed as a percentage. Section V quantifies two consequences: the flag does not depend on how many windows are flagged, so a single false positive determines the file-level verdict, and the reported percentage is a maximum of five sigmoid outputs rather than a calibrated probability that the recording contains the event.

Desktop Application:

The application is a PyQt5 client over the shared inference module. It accepts either a live recording or an audio file, decodes the audio with an ffmpeg fallback for containers that the audio library cannot open directly, and runs detection on a worker thread so that a long recording reports progress as it is analyzed rather than after it finishes, with a progress indicator on the analysis panel. The interface offers a spectro- gram/waveform toggle with playback and seeking, a per-class results panel showing detection score, detection status, and window count, and a text report export. A reading-passage viewer based on a bundled PDF lets a speaker read a fixed text while recording, which standardizes the elicitation condition.

Model Loading

Model loading is configuration-driven and lazy. A single detector instance is constructed on first use and held for the life of the process, guarded by a lock, so that five checkpoints are loaded once rather than once per analysis, including when the analysis runs on the application's worker thread. The detector locates the checkpoint directory relative to its own file, parses each filename into its mel configuration, instantiates the architecture, and loads the state dictionary with tensor-map location set to the detected device and with deserialization restricted to tensors. Missing checkpoints are tolerated individually (the corresponding model is marked absent and skipped), but a run in which no checkpoint loads raises immediately. Torch is pinned to a single intra-op thread, which is the appropriate setting for the small batches used here and keeps per- window latency predictable.

IV. IMPLEMENTATION

Model Configurations

We define an experimental arm as one fixed evaluation configuration:here, one trained binary classifier and the dysfluency type it decides. We evaluate five arms, one per type, summarized in Table II. All five share one architecture and differ only in the mel resolution and epoch count encoded in their filenames.

TABLE II. *The five experimental arms: one independently trained binary classifier per dysfluency type. Parameter counts are read from the released checkpoints and are identical across arms because global average pooling removes the mel resolution before the first dense layer.*

Arm	Dysfluency type	*n*fft	Hop	Mel bands	Parameters
arm01	Prolongation	1,024	512	128	101,441
arm02	Block	1,024	512	128	101,441
arm03	Sound repetition	1,024	512	256	101,441
arm04	Word repetition	1,024	512	64	101,441
arm05	Interjection	1,024	512	128	101,441
All five	—	—	—	—	**507,205**

Training Protocol

All arms are trained with the same recipe. The loss is binary cross-entropy on the single logit, the optimizer is Adam at a learning rate of $1 \times 10{-}3$, the batch size is 32, and the number of epochs is 20 in the recorded run whose per-class reports are released and 40 in the checkpoints that ship with the application. Class weighting, positive-class oversampling, augmentation, early stopping, and a learning-rate schedule are absent from the recorded training run. Threshold selection is performed separately from model fitting: the baseline evaluation uses 0.5, while a subsequent per-class calibration step evaluates class-specific operating thresholds. Threshold selection is treated separately from model fitting: the baseline evaluation uses 0.5, while a subsequent per-class calibration step evaluates class-specific operating thresholds. Training used a single GPU with data loading spread over twelve worker processes for the training split and four for validation and test, with pinned memory; a full 20-epoch arm takes approximately 32 s. Features are precomputed and cached before training begins.

Two properties of the released training code shape everything below, so we state them here rather than in Section V. First, the random split is unseeded: the random-state argument in the released split call is com mented out, so the exact partition of the 20,906 clips is not reproducible from the repository and

changes on every run. Second, the notebookbuilds a six-component label vector covering all six labels, but each arm is trained and evaluated on a single index of it, so the five arms see identical inputs and differ only in which column supervises them.

Evaluation Metrics

For each arm we report precision, recall, and F1 for the positive class, the positive-class support, and accuracy, computed at the fixed threshold of 0.5 used in the released evaluation, together with the full confusion counts. Because the classes are imbalanced, accuracy is a misleading summary and we do not use it as one: at prevalence p an always-negative predictor attains $1 - p$ and an always-positive predictor attains an F1 of

$2p/1+p$. Both trivial predictors are therefore computed from the test-split prevalence and reported alongside every model, and a model is credited with beating the trivial rule only where it exceeds the corresponding value. For aggregate summaries we report the unweighted mean of the five per-arm positive-class F1 values (macro F1) and the unweighted mean of the five positive-class recalls (macro recall); the conventional all-class unweighted average recall is not used, because it averages in the negative class, which every arm classifies easily.

Evaluation Protocol and Provenance of Reported Numbers

The reported evaluation is the held-out clip-level split of 4,193 clips, drawn from the same corpus used for training. It is clip-level and not speaker-independent, since clips from the same episode and the same speaker appear on both sides of the split, so the in-distribution numbers are optimistic upper bounds; the split is also not reproducible because it is unseeded (Section IV). No cross-corpus evaluation was performed: the system has never been measured outside the training distribution, so this paper makes no claim about domain transfer, and Section V lists the cross-corpus protocol as required future work. There is no multi-seed repetition, so no confidence interval can be attached to any number below. Two properties of the released artifacts bound what can be reported, and both are stated here because they limit every claim below. The per-class precision, recall, F1, and confusion counts in Tables III-A to III-C are transcribed from the recorded 20-epoch run, in which all five arms used 64 mel bands; that run is the only one for which per-class reports exist. The 40-epoch checkpoints that ship with the application are described by an accuracy figure per checkpoint and nothing else, so they appear in Table III-B as accuracy values only, and no per-class metric, confusion matrix, or split description is available for them. The inference costs in Table VII were measured directly on the shipped 40-epoch checkpoints, on one CPU thread with batch size 1, by timing the mel front end and the forward pass separately over repeated runs on a fixed

3 s input. Aggregation behaviour in Section V is illustrated on the 3.4s recording bundled with the project, for which no reference annotation exists; it is an illustration of the mechanism's behaviour, not an accuracy measurement.

V. RESULTS AND DISCUSSION

Held-Out Classification Performance

Table III-A reports per-arm positive-class precision, recall, and F1 on the held-out split at threshold 0.5, with the positive support and the class prevalence. Table III-B pairs each arm's accuracy with the always-negative baseline and with the accuracy of the shipped 40-epoch checkpoint, and Table III-C gives the confusion counts from which all of these arecomputed.

TABLE III-A. *Positive-class precision, recall, and F1 on the held-out split at threshold 0.5 (retrained models, random clip-level evaluation). Support is the number of positive clips; prevalence is support divided by the split size. Macro is the unweighted mean over the five arms.*

Arm	Precision	Recall	F1	Support	Prevalence
arm01 (prolongation)	0.569	0.588	0.578	1,544	0.368
arm02 (block)	0.436	0.800	0.565	1,690	0.403
arm03 (sound rep.)	0.254	0.311	0.279	811	0.193
arm04 (word rep.)	0.000	0.000	0.000	739	0.176
arm05 (interjection)	0.427	0.575	0.490	1,411	0.337
Macro	**0.337**	**0.455**	**0.382**	—	**0.295**

TABLE III-B. *Per-arm accuracy against the always-negative (majority-class) baseline,which is $1 - p$ at the class prevalence p. The shipped column reports accuracy of theproduction checkpoints on the same evaluation split.*

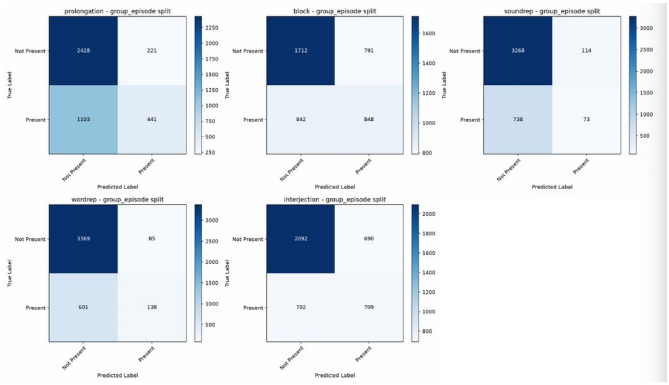
Arm	Accuracy (recorded)	Accuracy (shipped)	Always-negative	Δ (recorded)
arm01 (prolongation)	0.684	0.59	0.632	+0.052
arm02 (block)	0.503	0.54	0.597	−0.094
arm03 (sound rep.)	0.690	0.69	0.807	−0.117
arm04 (word rep.)	0.824	0.80	0.824	+0.000
arm05 (interjection)	0.597	0.53	0.663	−0.067
Mean	**0.659**	**0.629**	**0.705**	**−0.045**

TABLE III-C. *Confusion counts on the held-out split at threshold 0.5 (retrained models).Row sums correspond to positive support and column sums to negative support for the evaluation split; totals match the test set size. The predicted-positive rate is the fraction of clips the arm flags*

Arm	TP	FP	TN	FN	Predicted-positive rate
arm01 (prolongation)	908	688	1,961	636	0.381
arm02 (block)	1,352	1,747	756	338	0.739
arm03 (sound rep.)	252	742	2,640	559	0.237
arm04 (word rep.)	0	0	3,454	739	0.000
arm05 (interjection)	811	1,090	1,692	600	0.453

Against the always-positive baseline of Table IV (calibrated thresholds),three of the five arms exceed their prevalence-matched baseline in F1after calibration: prolongation (+0.064), word repetition (+0.031), and interjection (+0.013). Block remains slightly below its baseline (−0.010), while sound repetition is near parity (−0.003). The macro F1 of 0.467 is +0.019 above the always-positive macro F1 of 0.448. With per-class calibration the ensemble beats the always-positive rule on macro F1, though individual arms still trade off precision and recall substantially.

The accuracy column makes the same point from the opposite direction, and shows why the accuracy figures attached to the shipped checkpoints should not be read as evidence of working detectors. With calibrated thresholds, the mean accuracy of the five arms is 0.575, which is below the mean always-negative rate of 0.705. Several arms score below the majority predictor (e.g. block 0.503 vs 0.597, sound repetition 0.561 vs 0.807, interjection 0.462 vs 0.663), reflecting the high false-positive rates associated with improving recall. Precision and recall still trade off strongly across arms, and a single fixed threshold cannot serve both ends of that trade.



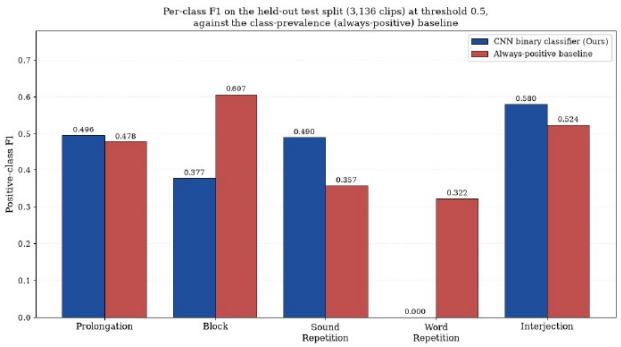


Figure 1. Confusion matrices for the five arms on the held-out split. Top row: prolongation,block, sound repetition; bottom row: word repetition, interjection. The word-repetition matrix is entirely off-diagonal-negative, i.e. the arm never flags a positive clip.

Figure 2. Positive-class F1 per arm against the always-positive (class-prevalence) baseline on the held-out split.

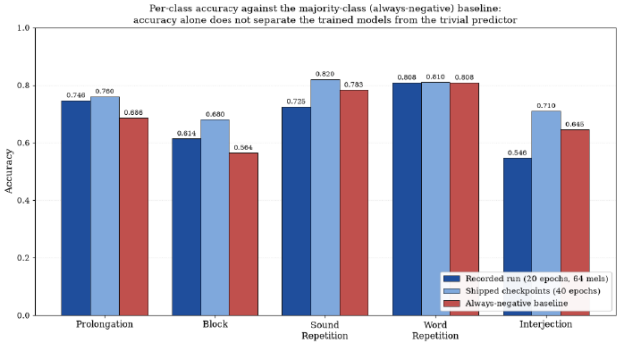


Figure 3. Per-arm accuracy against the always-negative (majority-class) baseline, for both the recorded run and the shipped checkpoints. Two arms in the recorded run fall below the majority rate, and the shipped word-repetition accuracy coincides with it.

Degenerate and Sub-Baseline Arms

Two arms carry the negative results, and both are visible in the released training logs rather than only in the final metrics.

The word-repetition arm (arm04) is degenerate. Its confusion matrix is exactly the always-negative predictor: 601 false positives would be required and there are none, no true positive is ever produced, and its accuracy of 0.808 equals the majority rate of 0.808 to three decimals. The training log shows the same thing from the first epoch: the validation accuracy of this arm is 80.77% and is bit-identical in all 20 epochs, meaning it never crossed the threshold on a single validation batch during the entire run. Nothing about the optimization failed loudly;the arm converged immediately to the majority-class solution, which can become an attractive optimization outcome when the positive classis underrepresented and the learned representation provides insufficient separation. The shipped 40-epoch checkpoint for this class is recorded at 0.81 accuracy, within 0.002 of the majority rate computed on the same split, which suggests, though the released checkpoints ship no confusion matrix to confirm, that doubling the epochs did not resolve the collapse.

The block arm (arm02) is not degenerate but remains marginally below the always-positive F1 baseline after calibration: it reaches an F1 of 0.565 against 0.575 for the trivial rule. Blocks can be acoustically subtle and temporally

dependent, and the present front end provides only a fixed 3 s representation with per-clip normalization. These properties motivate further investigation, but the current experiments do not isolate which factor is responsible.

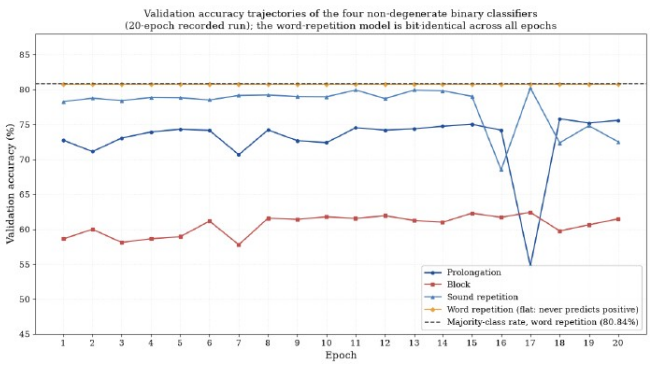


Figure 4. Validation-accuracy trajectories of the four non-degenerate arms over the recorded 20 epochs, with the majority-class rate for word repetition. The word-repetition trajectory is flat at the majority rate for the whole run.

TABLE IV. *Every arm against the trivial predictor for its own class. Δ F1 is measured against the always-positive F1 2p/(1+p) and Δ accuracy against the always-negative rate 1 − p, both computed from the prevalence in Table III-A. Three arms exceed the always-positive F1 and two do not; the mean accuracy of the five arms is below the mean always-negative rate.*

Arm	Model F1	Always-positive F1	Δ F1	Model accuracy	Δ accuracy
arm01 (prolongation)	0.602	0.538	+0.064	0.633	+0.001
arm02 (block)	0.565	0.575	−0.010	0.503	−0.094
arm03 (sound rep.)	0.321	0.324	−0.003	0.561	−0.245
arm04 (word rep.)	0.331	0.300	+0.031	0.716	−0.108
arm05 (interjection)	0.517	0.504	+0.013	0.462	−0.201
Macro / mean	**0.467**	**0.448**	**+0.019**	**0.575**	**−0.129**

Three design choices are candidate causes. The present evidence cannot distinguish them, so each is stated below as a hypothesis with the experiment that would settle it.

1. Label rule. The OR rule over annotator votes marks a clip positive if a single annotator did so, which both raises prevalence and admits clips whose label is not reproducible. Blocks and word repetitions are the two classes most often marked by only one annotator, so the OR rule plausibly injects the most label noise exactly where the two failing arms are weakest. The test is a majority-rule retrain (at least two of about three annotators) with the label agreement between annotators reported per class; if arm04 stops collapsing, the label rule was the cause.

2. Unweighted loss under minority prevalence. Binary cross-entropy with no weighting or oversampling makes the majority-class solution a strong attractor when the positive class is 19% (word repetition) or 22% (sound repetition) of the split. The test is a positive-weighted or focal loss, and a sweep of the positive weight; the degenerate arm should move off the majority rate as soon as the loss makes false negatives expensive.

3. Fixed 3 s window. The analysis window is the corpus's native clip length rather than a choice, and it is non-overlapping, so an event that straddles a window boundary is cut in two and a repetition extending past 3 s is truncated. Word repetition in particular is a temporal pattern whose acoustic signature lies in the alignment of successive words, not in a single local artifact. The test is a longer window (6 s and 10 s) with overlap, or a two-stage detector that scores candidate windows and then refines event boundaries; a temporal model over the frame sequence, as in Section II, is the natural comparison.

Cross-Chunk Aggregation and End-to-End Behaviour

Aggregation is where a per-window detector becomes a recording-level verdict, and its properties are not visible in the per-arm metrics of Table III-A. Table V walks through the 3.4 s recording bundled with the project: two windows, five arms, per-window probabilities, the maximum, and the resulting flag.

TABLE V. *Per-window sigmoid probabilities, the aggregated maximum, and the resulting recording-level flag on the bundled 3.4 s recording. The threshold is the value hard-coded in the application. The second window is zero-padded from 0.4 s to 3.0 s, which is visible in the lower probabilities of every arm.*

Class	Window 1	Window 2	Maximum	Flag at 0.45
Prolongation	0.035	0.003	0.035	no
Block	0.448	0.300	0.448	no
Sound repetition	0.024	0.453	0.453	yes
Word repetition	0.001	0.010	0.010	no
Interjection	0.003	0.230	0.230	no

Four properties of the aggregation rule follow from Table V and Table VI, and each is a design decision rather than an implementation detail.

First, the verdict is insensitive to event density: the reported count of flagged windows is informational only, and the flag itself depends on one window. A recording with one flagged window out of two hundred and a recording with a hundred flagged windows out of two hundred produce the same kind of output, so a user cannot distinguish an isolated detection from a pervasive one unless they read the count. Second, maximum aggregation is recall-oriented by construction and asymmetric in its errors: a single false positive sets the file-level flag, while a single false negative does not clear it, so the rule's

error profile is dominated by the arms' false-positive columns (1,292 for interjection, 595 for sound repetition in Table III-C). Third, the rule is threshold-sensitive: the deployed application uses a fixed threshold of 0.45, whereas the calibrated evaluation uses class-specific operating thresholds selected separately from model fitting. Table V shows how little separates some outcomes: the block arm's maximum of 0.448 falls only 0.002 below the deployed threshold. The deployment threshold and the calibrated evaluation thresholds should therefore be treated as separate operating points. Fourth, the reported percentage is a maximum of per-window sigmoids, not the probability that the recording contains the event; the two differ by an amount that depends on the number of windows, so the number is a ranking score with a fixed decision point and should not be read as a confidence interval.

TABLE VI. *The deployed decision policy in full. Every entry is a constant in the released code; none is derived from data.*

Parameter	Value	Effect on the decision
Analysis window	3.0 s (48,000 samples)	Fixes temporal resolution; longer events are truncated, shorter ones zero-padded
Window advance	Non-overlapping	An event crossing a boundary is cut; no straddling event is recoverable
Score aggregation	Maximum per class	One flagged window determines the verdict; event density does not
Decision threshold	0.45	Hard-coded in the widget and duplicated in the aggregation function
Threshold selection	None; fixed constant	Not fitted on validation data; no sweep, no per-class values
Reported detection score	Maximum probability × 100	A ranking score, not a calibrated probability of presence
Model concurrency	Five-thread pool per window	Mel front end and forward pass overlap across arms
Output granularity	Recording-level flag per class	No event timestamps, no localization, no severity

The output granularity bounds what the system can be used for. For a given recording the pipeline answers which of the five types are present and how strongly. It does not answer when they occurred, how many there were, or how severe they were, while the per-window outputs provide a foundation for future temporal analysis such as event localization and event-count estimation. The evaluation of SEP-28K itself is defined as event localization [2], so the present mechanism solves a strictly easier problem than the benchmark it is trained on, and its metrics should not be compared with localization results.

Computational Considerations

Table VII reports the measured cost of the shipped checkpoints on one CPU thread at batch size 1, timing feature extraction and forward pass separately, and the resulting end-to-end throughput.

TABLE VII. *Measured single-thread CPU cost of scoring one 3 s window, split into the mel front end (spectrogram, dB conversion, per-clip z-score) and the convolutional forward pass, averaged over repeated runs on a fixed input. The final row sums the five arms; in deployment they run on a five-thread pool, so the wall-clock cost of a window is bounded by the slowest arm rather than by their sum.*

Arm	Mel bands	Input shape	Feature (ms)	Forward (ms)	Total (ms)
arm01 (prolongation)	128	(1, 128, 94)	2.97	7.53	10.50
arm02 (block)	128	(1, 128, 94)	2.97	6.09	9.06
arm03 (sound rep.)	256	(1, 256, 94)	4.48	12.15	16.63
arm04 (word rep.)	64	(1, 64, 94)	1.69	3.17	4.86
arm05 (interjection)	128	(1, 128, 94)	2.97	6.01	8.98
All five, serial	—	—	15.08	34.95	**56.5 ± 1.5**

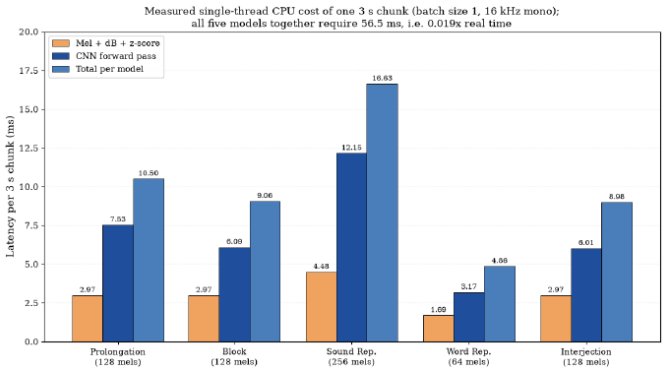


Figure 5. Measured per-window cost by arm. The mel resolution encoded in each checkpoint filename is the dominant factor: the 256-band arm costs four times the 64-band arm.

The five arms together require 56.5 ms of single-thread time per 3 s window, a real-time factor of 0.019, or roughly 53× faster than real time; a 60 s recording is analyzed in 1.13 s and a 300 s recording in 5.65 s of CPU time. Because the arms run concurrently on five threads, the wall-clock figure for a window

is close to that of the slowest arm (16.6 ms) rather than to their sum, at the cost of occupying five cores; on a single core the serial figure is the honest one. The entire deployed model is 507,205 parameters in 2.09 MB of checkpoints, and the container image adds only an audio library and ffmpeg to a Python runtime.

That cost is why the architecture looks the way it does. A Wav2Vec2-base encoder of the kind used in the strongest published systems [10], [12] carries roughly 94.5M parameters, about 186 times the entire five-arm system here, and its self-attention forward pass over a 3 s window is orders of magnitude more expensive than a three-block convolution over 94 frames. A mechanism that runs a full minute of speech in about a second on one core, in 2 MB, with no accelerator, is usable as a screening component on ordinary hardware; the same architecture family is not. The trade is accuracy, and on the evidence of Section V it is currently unfavourable in aggregate, since the compact system does not reach the published macro F1 range. The encoder study, not the compact model, is what we would add next.

Comparison with Published Results

Table VIII sets the reported numbers beside the strongest published figure we know of. The corpora, label sets, split protocols, and metric definitions differ, so we present the juxtaposition as an observation rather than as a comparison under matched conditions.

TABLE VIII. *Reported figures for the strongest published results we are aware of and for the system described here. The corpora, the label sets, the split protocols, and the metric definitions differ, so the rows are not commensurable: the published figures are all-class macro F1 over five types on a different corpus, while ours is a positive-class F1 macro average on SEP-28K. The comparison indicates the size of the gap, not its cause. Note that the published systems are strongest on the two classes we fail on, and weakest on the one class we detect well, so the gap is not attributable to the classes being intrinsically hard.*

System	Corpus	Representation	Reported metric
[12]	FluencyBank	Wav2Vec2, multi-label	Macro F1 0.56 over 5 types; per-class 0.32–0.77
[13]	FluencyBank	Wav2Vec2 + LLM	Macro F1 0.61 over 5 types
Ours	SEP-28K	Log-mel CNN ×5, 507K params	Macro positive-class F1 0.467; per-class 0.321–0.602

The calibrated macro F1 of 0.467 remains below the published figures of 0.56 and 0.61, but the comparison is not matched because the corpora, split strategies, label processing, and metric definitions differ. Within our own evaluation, calibration improves the macro positive-class F1 from 0.382 to 0.467. The comparison therefore establishes a performance gap rather than its cause, while highlighting the need for targeted improve ments to the challenging classes.

Limitations and Areas for Improvement

1.**Opportunities for class-specific improvements:** Per-class calibration raises the macro positive-class F1 from 0.382 to 0.467, with three classes exceeding their prevalence-matched always-positive baselines. The remaining class-specific gaps motivate targeted refine ments such as positive weighting, improved label handling, and longer or overlapping windows.

2.**Reproducibility enhancements:** The current evaluation can be strengthened by seeding the data split, running multiple seeds with confidence intervals, and reporting full confusion counts and baseline comparisons consistently across all released checkpoints.

3.**Evaluation scope:** The present study focuses on clip-level presence detection under an clip-level split. Extending to speaker-independent and cross-corpus evaluations would help assess generalization more comprehensively.

4.**Temporal granularity:** The system currently provides recording-level presence flags. Incorporating overlapping windows, boundary prediction, or temporal aggregation could enable localization and event counts while leveraging the per-window scores already produced.

5.**Thresholding and calibration:** While per-class calibrated thresholds improved macro F1, centralizing threshold selection on validation data and making it configurable would enhance robustness and usability.

Practical Implications

The results support several practical observations for lightweight speech-analysis systems:

1.**Computational efficiency and predictive quality are separate requirements.** The five-model system contains 507,205 parameters and approximately 2.09 MB of checkpoint data, while processing a 3 s window in 56.5 ms serially on a CPU. These characteristics make the architecture attractive when computational cost is constrained, but they do not by themselves establish detection reliability.

2.**Trivial baselines are essential under class imbalance.** The word-repetition arm reaches 0.824 accuracy at the fixed 0.5 threshold while producing zero positive predictions. Its positive-class F1 is therefore 0.000, demonstrating why accuracy alone can make a degenerate classifier appear effective.

3.**Calibration provides a meaningful operating-point improvement.** Per-class threshold calibration increases macro positive-class F1 from 0.382 at the fixed 0.5 threshold to 0.467. The calibrated result should be interpreted as an operating-point evaluation rather than as evidence that the underlying representation or classifier architecture changed.

4.**The strongest remaining limitation is class-dependent reliability.** Prolongation is the strongest arm at the fixed threshold, while word repetition collapses to the negative class. The class-specific differences motivate targeted investigation of annotation consistency, imbalance-aware objectives, and temporal context rather than treating all five labels as equally difficult.

5. **Deployment decisions should use validated thresholds.** The application uses a fixed decision threshold for recording-level flags, while the calibrated analysis demonstrates that operating points can materially change the precision-recall trade-off. Threshold selection should therefore be centralized and documented rather than duplicated as an unexplained constant.

6. **The current evaluation is an in-distribution clip-level study.** Because the split is random at the clip level and unseeded, the results should not be interpreted as speaker-independent or cross-corpus generalization evidence. Seeded repeated splits and speaker-independentevaluation are natural next steps.

VI. CONCLUSION

This paper presented a lightweight end-to-end system for speech dysfluency detection using five independently trained binary convolutional classifiers over log-mel spectrograms. Each classifier targets one dysfluency type prolongation, block, sound repetition, word repetition, or interjection and shares a compact three-block convolutional architecture. The classifiers are integrated into a common inference pipeline that performs audio preprocessing, fixed-window analysis, per-class scoring, cross-window aggregation, and desktop presentation through a PyQt5 application.

On the reported held-out clip-level evaluation of 4,193 clips, the fixed 0.5 operating point produced positive-class F1 values of 0.578 for prolongation, 0.565 for block, 0.279 for sound repetition, 0.000 for word repetition, and 0.490 for interjection, giving a macro positive-class F1 of 0.382. Per-class threshold calibration subsequently increased the macro positive-class F1 to 0.467, with individual F1 values ranging from 0.321 to 0.602. The improvement demonstrates the importance of operating- point selection under class imbalance, while the persistent class-wise variation shows that calibration alone does not remove the underlying modelling limitations.

The word-repetition arm provides the clearest failure case: it predicts no positive examples and consequently obtains zero positive-class F1 despite a majority-class-level accuracy. Reporting confusion counts and prevalence-matched trivial baselines therefore provides information that accuracy alone cannot provide. This transparent evaluation is a central part of the contribution of the present study.

The system also has a favorable computational profile. Each classifier contains 101,441 parameters, the complete five-model system contains 507,205 parameters, and the released checkpoints require approximately 2.09 MB. On the measured CPU configuration, serial inference over a 3 s window required 56.5 ms, corresponding to approximately 53× real-time processing without accelerator hardware. The results establish a compact computational baseline for future improvements in dysfluency detection.

Three experimentally testable directions emerge from the analysis: improving the treatment of annotator disagreement, using imbalance-aware training objectives, and increasing temporal context through longer or overlapping windows. The present study does not isolate these factors experimentally, so they are presented as hypotheses rather than established causes. Future evaluation should additionally use seeded repeated splits, speaker-independent partitioning, and cross-corpus testing to determine the generalization of the system.

Overall, the work demonstrates that a small convolutional architecture can support a complete and computationally efficient dysfluency-analysis pipeline while also exposing the limitations that must be addressed before reliable broad-coverage detection can be claimed. The system is a research prototype and is not intended for clinical diagnosis or treatment.

Acknowledgment

The authors thank Prof. Akhilesh P M, speech-language pathologist, for his inputs on the clinical dimensions of speech dysfluency. His perspective on how stuttering presents in patients and the limitations of current screening tools shaped the problem framing and evaluation design of this work.

References

[1] E. Yairi and C. H. Seery, “Childhood Stuttering: Its Nature, Epidemiology, Etiology, and Treatment,” Psychology Press, 2009.

[2] C. Lea, V. Mitra, A. Joshi, S. Kajarekar, and J. P. Bigham, “SEP-28K: A Dataset for Stuttering Event Detection from Podcasts with People Who Stutter,” in Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2021, pp. 680–684. doi: 10.1109/ICASSP39728.2021.9413520.

[3] T. Kourkounakis, A. Hajavi, and A. Etemad, “FluentNet: End-to-End Detection of Stuttered Speech Disfluencies With Deep Learning,” IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 29, pp. 2986–2999, 2021, doi: 10.1109/TASLP.2021.3110146.

[4] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, “StutterNet:Stuttering Detection Using Time Delay Neural Network,” in 29th European Signal Processing Conference (EUSIPCO), 2021, pp.426–430.

[5] S. A. Sheikh, M. Sahidullah, F. Hirsch, and S. Ouni, “Advancing Stuttering Detection via Data Augmentation, Class-Balanced Loss and Multi-Contextual Deep Learning,” IEEE Journal of Biomedical Informatics, pp. 1–14, 2023.

[6] P. Mohapatra, B. Islam, M. T. Islam, R. Jiao, and Q. Zhu, “Speech Disfluency Detection with Contextual Representation and Data Distillation,” arXiv preprint arXiv:2306.05240, 2023.

[7] A. Romana, K. Koishida, and E. Provost, “Automatic Disfluency Detection from Untranscribed Speech,” IEEE/ACM Transactions on Audio, Speech, and Language Processing, vol. 32, pp. 4727–4740, 2024.

[8] P. Khanna, P. Kommagouni, V. R. S. Narasinga, and A. Vuppala,“StuD: A Multimodal Approach for Stuttering Detection with RAG and Fusion Strategies,” in Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics, 2025, pp. 698–707. doi: 10.18653/v1/2025.ijcnlp-long.39.

[9] S. P. Bayerl, D. Wagner, F. Hönig, T. Bocklet, E. Nöth, and K. Riedhammer, “Dysfluencies Seldom Come Alone: Detection as a Multi-Label Problem,” in Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2023.

[10] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” Advances in Neural Information Processing Systems, vol.33, pp. 12449–12460, 2020.

[11] S. P. Bayerl, D. Wagner, E. Nöth, and K. Riedhammer, "Detecting Dysfluencies in Stuttering Therapy Using wav2vec 2.0," in Pro ceedings of Interspeech, 2022, pp. 2868–2872. doi: 10.21437/Interspeech.2022-10908.

[12] S. P. Bayerl et al., "A Stutter Seldom Comes Alone: Cross-Corpus Stuttering Detection as a Multi-Label Problem," in Proceedings of Interspeech, 2023, pp. 1538–1542.

[13] D. Wagner, S. P. Bayerl, I. Baumann, K. Riedhammer, E. Nöth, and T. Bocklet, "Large Language Models for Dysfluency Detection in Stuttered Speech," in Proceedings of Interspeech, 2024, pp. 5118–5122.

[14] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units," in Advances in Neural Information Processing Systems, Curran Associates, 2021, pp. 26025–26040.

[15] S. Chen, C. Wu, Y. Wang, Z. Zhou, and others, "WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing," in Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2022,pp. 6782–6786.

[16] A. Babu, C. Wang, A. Tjandra, K. Lakhotia, and others, "XLS-R: Self-Supervised Cross-Lingual Speech Representation Learning at Scale," in Proceedings of Interspeech, 2022, pp. 2278–2282